Financial Calculators: Test Cases

Today's job

Today we're Quality Assurance Testers (QA Testers). Before a program goes in front of a customer, someone other than the developer tests it. You're going to test your **partner's** calculators, and your partner is going to test yours.

Steps

1. **Swap repo links.** Paste your GitHub repo URL into the Zoom chat for your partner, and grab theirs.

2. **Clone your partner's repo** onto your computer. In your terminal, go to a folder outside your own project and run:

```
git clone <your-partner's-repo-url>
```

3. **Open the project and run it.** If it won't compile or run, that's your first bug report. Tell your partner and help each other get it running before you test.

4. **Go through every test case.** Type in the numbers from each row exactly as shown and compare the output to the *Expected* columns.

5. **Keep track of Pass or Fail.** A plain text file or Notepad is fine. A case passes if it matches the *Expected* column (see the penny rule below). Otherwise it fails.

6. **Report back.** When you're done, go over your results with your partner. For every Fail, tell them the inputs you typed and what the program printed.

> **Hands off their code.** As the tester, you find bugs. You don't fix them. Your partner fixes their own code, then you re-run the tests that failed.

> **The penny rule.** A penny off is okay. Java can't store most decimal numbers perfectly, so an answer might be off by a cent or two. That's normal and counts as a Pass. If it's off by whole dollars, it's a Fail. Common causes: using `float` instead of `double`, forgetting to divide the rate by 100 (1.75% needs to become 0.0175), or a mistake in the formula.

Calculator 1: Mortgage

Principal	Rate	Years	Expected monthly payment	Expected total interest
$53,000	7.625%	15	$495.09	$36,115.99
$200,000	6.5%	30	$1,264.14	$255,088.98
$150,000	3%	15	$1,035.87	$36,457.04
$25,000	5%	5	$471.78	$3,306.85

Calculator 2: CD future value

Deposit	Rate	Years	Expected future value	Expected interest earned
$1,000	1.75%	5	$1,091.44	$91.44
$10,000	5%	1	$10,512.67	$512.67
$2,500	2.25%	3	$2,674.57	$174.57
$5,000	4%	10	$7,458.96	$2,458.96

Note on the first CD case. The workbook lists $1,092.62 for this example, but its own daily-compounding formula gives $1,091.44. If the program shows $1,091.44, that's a Pass.

Calculator 3: Annuity present value

Monthly payout	Rate	Years	Expected present value
$3,000	2.5%	20	$566,141.46
$1,000	5%	10	$94,281.35
$500	6%	5	$25,862.78

Bonus: Try to Break It

Every test above uses nice, sensible numbers. Real users don't always do that. They make typos, they misread the prompt, and some of them are just curious what happens. A good QA tester thinks like all of those people.

Try the inputs below on your partner's program. **Before you hit Enter, predict what will happen** and say it out loud to your partner. Then run it and see if you were right.

> **Crashes are expected here.** We haven't learned how to handle bad input yet (that's coming with `try`/`catch` and input validation). Nobody loses points for these. The goal is to notice the problems so you know what we're solving later.

Calculator	Try this
Any	Type `abc` when it asks for a number
Any	Type `7.5%` (with the percent sign) for the rate
Any	Just press Enter without typing anything
Any	Type `15.5` for years
Mortgage	Interest rate of `0`
Mortgage	Years of `0`
Any	Principal or deposit of `0`
Any	A negative amount, like `-50000`
Any	A negative rate, like `-5`
Any	Rate typed as a decimal, `0.05` instead of `5`
Any	A huge number, like `99999999999999`
Your idea	Anything else a real user might type. Get creative.

Talk it over with your partner

1. Which inputs made the program crash? Which ones didn't crash but gave an answer that makes no sense? Which is worse for a customer?

2. Did anything print `NaN` or `Infinity`? Look at the formula. Why might that happen?

3. Someone typed 0.05 meaning 5%. The program didn't crash, but was the answer right? Whose fault is that: the user's or the program's?

4. Is a 0% interest rate a real thing? (Hint: some car dealers offer it.) Should the mortgage calculator handle it?

5. If you could change one thing about the prompts your program shows the user to prevent mistakes, what would it be?